

Interactions API

The Interactions API is the best way to build with Gemini models and agents. As of June 2026, it is Generally Available and recommended for all new projects. While it is now considered legacy, the original [generateContent](/gemini-api/docs/generate-content/text-generation) (/gemini-api/docs/generate-content/text-generation) API remains fully supported.

Why use the Interactions API?

- **Universal interface for all applications:** Designed as the standard interface for every use case, including single-turn text generation, multimodal understanding, structured outputs, tool orchestration, and agentic workflows.
- **Single API for models and agents:** One unified endpoint and pattern for calling standard Gemini models as well as specialized agents directly (such as Deep Research and custom managed agents).
- **New capabilities out of the box:** Features like optional server-side conversation state using `previous_interaction_id`, observable execution steps for debugging and UI rendering, and [background execution](/gemini-api/docs/background-execution) (/gemini-api/docs/background-execution) for long-running tasks using `background=true`.
- **Lower cost with higher cache hit rates:** When using multi-turn conversations, optional server-side state management enables more efficient context caching across turns, reducing token costs.
- **Where new features launch:** Going forward, all new models, multimodal capabilities, tools, and agentic features will launch on the Interactions API.

By default, the Interactions API stores requests so you can leverage the server-side state management features by using `previous_interaction_id`. You can opt into stateless behavior by setting `store=false`. See the [data retention](#) (#data-storage-retention) section for details.

Get started

- **Set up your coding agent:** Connect to the **Gemini Docs MCP** and install the `gemini-api-dev` skill to give your assistant direct access to the latest developer docs and best practices. For detailed steps, see the [Set up your coding agent guide](#) (/gemini-api/docs/coding-agents)
- **Migrate from `generateContent`:** If you have an existing integration, follow the [Migration guide](#) (/gemini-api/docs/migrate-to-interactions) to transition to the Interactions API.
- **Get started:** Follow the steps in the [Interactions API Get started guide](#) (/gemini-api/docs/get-started).

Feature guides

Explore the specific capabilities of the Interactions API through these guides. You can use the toggle on these pages to switch between `generateContent` and Interactions API:

- [Text generation](#) (/gemini-api/docs/text-generation)
- [Image generation](#) (/gemini-api/docs/image-generation)
- [Image understanding](#) (/gemini-api/docs/image-understanding)
- [Audio understanding](#) (/gemini-api/docs/audio)
- [Video understanding](#) (/gemini-api/docs/video-understanding)

- [Document processing](/gemini-api/docs/document-processing) (/gemini-api/docs/document-processing)
- [Function calling](/gemini-api/docs/function-calling) (/gemini-api/docs/function-calling)
- [Structured output](/gemini-api/docs/structured-output) (/gemini-api/docs/structured-output)
- [Deep Research agent](/gemini-api/docs/deep-research) (/gemini-api/docs/deep-research)
- [Flex inference](/gemini-api/docs/flex-inference) (/gemini-api/docs/flex-inference)
- [Priority inference](/gemini-api/docs/priority-inference) (/gemini-api/docs/priority-inference)

How the Interactions API works

The Interactions API centers around a core resource: the **Interaction** (/api/interactions-api#Resource:Interaction). An **Interaction** represents a complete turn in a conversation or task. It acts as a session record, containing the entire history of an interaction as a chronological sequence of **execution steps**. These steps include model thoughts, server-side or client-side tool calls and results (like **function_call** and **function_result**), and the final **model_output**. The stored resource (retrieved via **interactions.get**) also includes **user_input** steps for full context, though the **interactions.create** response only returns model-generated steps.

When you make a call to **interactions.create** (/api/interactions-api#CreateInteraction), you are creating a new **Interaction** resource:

Python **JavaScript** (#javascript) **Java** (#java) **Go** (#go) **REST** (#rest)
(#python)

```
from google import genai

client = genai.Client()
```

```
interaction = client.interactions.create(
    model="gemini-3.8-flash",
    input="Tell me a short story about a time-traveling lighthouse."
)

print(interaction.output_text)
```

Server-side state management

You can use the `id` of a completed interaction in a subsequent call using the `previous_interaction_id` parameter to continue the conversation. The server uses this ID to retrieve the conversation history, saving you from having to resend the entire chat history:

PythonJavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#python)

```
from google import genai

client = genai.Client()

# 1. First turn
turn1 = client.interactions.create(
    model="gemini-3.8-flash",
    input="Hi, my name is Phil."
)

# 2. Second turn (chained using previous_interaction_id)
turn2 = client.interactions.create(
    model="gemini-3.8-flash",
    input="What is my name?",
```

```
    previous_interaction_id=turn1.id
)

print(turn2.output_text)
```

The `previous_interaction_id` parameter preserves only the conversation history (inputs and outputs) using `previous_interaction_id`. The other parameters are **interaction-scoped** and apply only to the specific interaction you are currently generating:

- `tools`
- `system_instruction`
- `generation_config` (including `thinking_level`, `temperature`, etc.)

This means you must re-specify these parameters in each new interaction if you want them to apply. This server-side state management is optional; you can also operate in stateless mode by sending the full conversation history in each request.

Data storage and retention

By default, the API stores all Interaction objects (`store=true`) in order to simplify use of server-side state management features (with `previous_interaction_id`), [background execution](/gemini-api/docs/background-execution) (using `background=true`) and observability purposes.

- **Paid tier:** The system retains interactions for **55 days**.
- **Free tier:** The system retains interactions for **1 day**.

If you don't want this, you can set `store=false` in your request. This control is separate from state management; you can opt out of storage for any interaction. However, note that `store=false` is incompatible with [background execution](/gemini-api/docs/background-execution) (/gemini-api/docs/background-execution) and prevents using `previous_interaction_id` for subsequent turns.

For Paid Tier projects, you can configure the retention window in [AI Studio](https://aistudio.google.com/logs) (https://aistudio.google.com/logs) to automatically mark logs for deletion from project storage after 7, 14, 28, or 55 days. A shorter retention may affect retrieval of past conversations.

You can delete stored interactions at any time using the `delete` (/api/interactions-api#deleteInteraction) method programmatically, which requires the interaction ID. You can also view and manage stored interactions logs, including deletion from project storage, in [AI Studio](https://aistudio.google.com/logs) (https://aistudio.google.com/logs).

After the retention period expires, your data will be deleted automatically.

Interactions objects are processed according to the [terms](/gemini-api/terms) (/gemini-api/terms).

View interactions in AI Studio

The API stores Interactions API requests executed with `store=true` for projects on the Paid Tier. You can view them directly from the [Logs page in Google AI Studio](/gemini-api/docs/www.aistudio.google.com/logs) (/gemini-api/docs/www.aistudio.google.com/logs). See the [Logs guide](/gemini-api/docs/logs-datasets) (/gemini-api/docs/logs-datasets) for more.

Best practices

- **Cache hit rate:** Implicit caching is supported in both stateful and stateless modes (see [Quickstart](/gemini-api/docs/get-started#4_multi-turn_conversations) (/gemini-api/docs/get-started#4_multi-turn_conversations)). Using `previous_interaction_id` (stateful) to continue conversations allows the system to more easily utilize implicit caching for the conversation history, which improves performance and reduces costs.

- **Mixing interactions:** You have the flexibility to mix and match Agent and Model interactions within a conversation. For example, you can use a specialized agent, like the Deep Research agent, for initial data collection, and then use a standard Gemini model for follow-up tasks such as summarizing or reformatting, linking these steps with the `previous_interaction_id`.

Supported models & agents

Model Name	Type	Model ID
Gemini 3.8 Flash	Model	<code>gemini-3.8-flash</code>
Gemini 3.7 Flash	Model	<code>gemini-3.7-flash</code>
Gemini 3.6 Flash	Model	<code>gemini-3.6-flash</code>
Gemini 3.5 Flash	Model	<code>gemini-3.5-flash</code>
Gemini 3.1 Pro Preview	Model	<code>gemini-3.1-pro-preview</code>
Gemini 3.5 Flash-Lite	Model	<code>gemini-3.5-flash-lite</code>
Gemini 3.1 Flash-Lite	Model	<code>gemini-3.1-flash-lite</code>

Model Name	Type	Model ID
Gemini 3 Flash Preview	Model	gemini-3-flash-preview
Gemini 2.5 Pro	Model	gemini-2.5-pro
Gemini 2.5 Flash	Model	gemini-2.5-flash
Gemini 2.5 Flash-lite	Model	gemini-2.5-flash-lite
Gemini 3 Pro Image	Model	gemini-3-pro-image
Gemini 3.1 Flash Image	Model	gemini-3.1-flash-image
Gemini 3.1 Flash TTS Preview	Model	gemini-3.1-flash-tts-preview
Gemma 4 31B IT	Model	gemma-4-31b-it
Gemma 4 26B MoE IT	Model	gemma-4-26b-a4b-it
Lyria 3.5	Model	lyria-3.5
Lyria 3 Clip Preview	Model	lyria-3-clip-preview

Model Name	Type	Model ID
Lyria 3 Pro Preview	Model	lyria-3-pro-preview
Deep Research Preview	Agent	deep-research-preview-04-2026
Deep Research Preview	Agent	deep-research-max-preview-04-2026
Antigravity Preview	Agent	antigravity-preview-09-2026

SDKs

You can use latest version of the Google GenAI SDKs in order to access Interactions API.

- On Python, this is `google-genai` package from `2.3.0` version onwards.
- On JavaScript, this is `@google/genai` package from `2.3.0` version onwards.
- On Go, this is `google.golang.org/genai` package.
- On Java, this is `com.google.genai:google-genai` package.

You can learn more about how to install the SDKs on [Libraries](/gemini-api/docs/libraries) (/gemini-api/docs/libraries) page.

Limitations

- **Remote MCP:** Gemini 3 does not support remote MCP, this is coming soon.
- **Multi-turn model compatibility:** When mixing different models in a conversation (either stateful or stateless), subsequent models must support the output modalities of the previous models as input. For example, if you generate an image using `gemini-3.1-flash-image`, you cannot continue that conversation with a model that doesn't accept image inputs (such as a text-only model or a music-generation model like Lyria).

The following features are supported by the `generateContent` (/gemini-api/docs/generate-content/text-generation) API but are **not yet available** in the Interactions API:

- **Batch API** (/gemini-api/docs/batch-api)
- **Automatic function calling (Python)**
(https://ai.google.dev/gemini-api/docs/function-calling?example=meeting#automatic_function_calling_python_only)
- **Explicit caching** (/gemini-api/docs/caching): Note that server-side implicit caching is available in the Interactions API via `previous_interaction_id`.
- **Safety settings** (/gemini-api/docs/safety-settings): Custom safety settings are not supported in the Interactions API.

Feedback

Your feedback is critical to the development of the Interactions API. Share your thoughts, report bugs, or request features on our [Google AI Developer Community Forum](https://discuss.ai.google.dev/c/gemini-api/4) (<https://discuss.ai.google.dev/c/gemini-api/4>).

What's next

- Try the [Interactions API quickstart notebook](https://colab.sandbox.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started_interactions_api.ipynb) (https://colab.sandbox.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started_interactions_api.ipynb).
- Learn more about the [Gemini Deep Research agent](/gemini-api/docs/deep-research) (/gemini-api/docs/deep-research).

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-09-23 UTC.